

3 Documento di Specifica dei Requisiti Software

3.1 Prospettiva del prodotto

BugBoard26 si colloca nel contesto degli strumenti di supporto allo sviluppo software, fornendo una piattaforma unificata per la gestione delle *issue*. Il sistema è concepito come un'applicazione indipendente, accessibile tramite interfaccia *web-based*, ma potenzialmente integrabile con altri strumenti di sviluppo.

Il prodotto adotta un'architettura client-server, in cui l'interfaccia utente comunica con un server centralizzato per la gestione dei dati relativi a utenti, issue, commenti e cronologie. Tutti i dati vengono conservati in un database relazionale per garantire consistenza e tracciabilità.

3.2 Funzionalità del sistema

Il sistema offrirà le seguenti funzionalità principali:

- autenticazione sicura tramite email e password;
- gestione delle utenze da parte degli amministratori, con possibile aggiunta di scadenze opzionali per la risoluzione dei bug;
- creazione delle issue con titolo, descrizione, tipologia, priorità e allegati;
- vista riepilogativa con possibilità di filtrare, ordinare e ricercare le issue secondo criteri multipli;
- assegnazione dei bug ai membri del team da parte degli amministratori, con relativo sistema di notifiche e area di visualizzazione personale;
- modifica dei bug nel rispetto del proprio livello di accesso;
- tracciamento della cronologia delle modifiche per i singoli bug.

Queste funzionalità sono formalizzate visivamente nello [Use Case UML](#) alla pagina seguente.

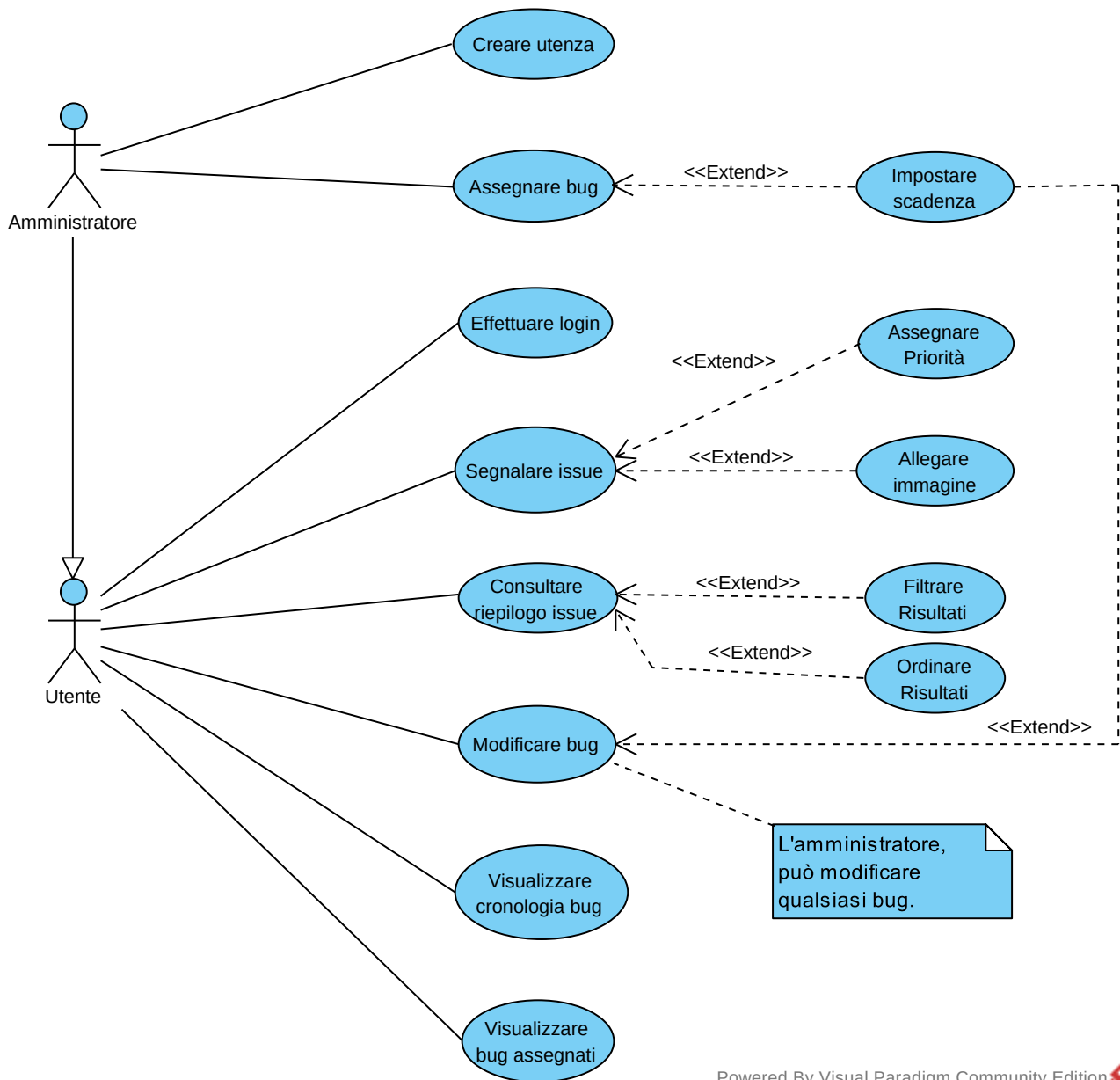
3.3 Caratteristiche degli utenti

Gli utenti del sistema sono profilati secondo un modello di controllo degli accessi basato sui ruoli, noto come Role-Based Access Control (RBAC), distinguendo due categorie gerarchiche:

Utente (Membro del team): rappresenta l'attore base autenticato. Possiede i diritti di lettura globale che gli consentono di consultare l'elenco di tutte le segnalazioni, filtrarle, ordinarle ed esaminarne il relativo storico delle modifiche. In scrittura è abilitato ad aprire nuove segnalazioni quali bug, feature, documentation e question. Può modificare lo stato e i dettagli esclusivamente dei bug di cui risulta formalmente assegnatario.

Amministratore (Admin): rappresenta un'utenza con privilegi di gestione avanzati che estende le capacità dell'Utente base. Detiene la responsabilità esclusiva di registrare nuove utenze, assegnare i bug ai vari membri del team per la risoluzione e definire le relative scadenze. Gode di diritti di modifica completi e incondizionati su qualsiasi issue o bug presente a sistema.

3.4 Use Case UML



Powered By Visual Paradigm Community Edition

Figura 1: Use Case UML

3.4.1 Descrizione testuale strutturata

USE CASE #1	Segnalare issue		
Goal in context	Segnalare agli altri membri del team una issue su un progetto		
Preconditions	L'utente deve aver effettuato il login		
Success End Conditions	Il sistema tiene traccia della segnalazione della issue, che viene creata con stato iniziale "TODO"		
Failure End Conditions	La issue non viene aggiunta		
Primary Actor	Utente		
Trigger	L'Utente clicca l'apposito bottone per segnalare issue		
Main Scenario	Step n.	Utente	System
	1	Clicca <i>Segnala issue</i>	
	2		Mostra M02
	3	Scrivo <i>Titolo, Descrizione, Progetto e Tipo</i>	
	4	Clicca <i>Ok</i>	
	5		Mostra M03
	6		Mostra M01
Extension #1 (clicca <i>Canc</i> e poi <i>Annulla</i>)	4a	Clicca <i>Canc</i>	
	5a		Mostra M04
	6a	Clicca <i>Annulla</i>	
	7a		Ritorna allo step 2 del main scenario
Extension #2 (clicca <i>Canc</i> e poi <i>Procedi</i>)	4b	Clicca <i>Canc</i>	
	5b		Mostra M04
	6b	Clicca <i>Procedi</i>	
	7b		Mostra M01
Extension #3 (specifica altre info)	3d	Inserisce anche priorità e allega immagine	
	4d	Clicca <i>Ok</i>	
	5d		Ritorna allo step 5 del main scenario
Extension #4 (Immagine non supportata)	4e		Mostra M05
	5e	Clicca <i>Ok</i>	
	6e		Ritorna allo step 3 del main scenario
Extension #5 (Errore generico)	4f		Mostra M06
	5f	Clicca <i>Ok</i>	
	6f		Ritorna allo step 3 del main scenario

USE CASE #2	Assegnare bug		
Goal in context	Assegnare un bug a un membro del team per la risoluzione e, opzionalmente, fissare una scadenza		
Preconditions	L'Amministratore deve aver effettuato il login. Deve esistere almeno un bug nel sistema.		
Success End Conditions	Il bug è assegnato all'utente scelto, il quale riceve una notifica. L'eventuale scadenza viene registrata.		
Failure End Conditions	L'assegnazione non viene completata. Il bug rimane nel suo stato precedente.		
Primary Actor	Amministratore		
Trigger	L'amministratore clicca sul bottone per assegnare uno specifico bug		
Main Scenario	Step n.	Admin	System
	1	Clicca <i>Assegna Bug</i>	
	2		Mostra M07
	3	Seleziona <i>Progetto, Bug</i> ed <i>Utente</i> dalla lista	
	4	Clicca <i>Ok</i>	
	5		Mostra M08 ed invia notifica all' <i>Utente</i>
	6		Mostra M01
Extension #1 (<i>Admin</i> sceglie di impostare scadenza)	3a	Inserisce data in <i>Scadenza</i>	
	4a		Ritorna allo step 5 del main scenario
Extension #2 (clicca <i>Canc</i> e poi <i>Procedi</i>)	4b	Clicca <i>Canc</i>	
	5b		Mostra M09
	6b	Clicca <i>Procedi</i>	
	7b		Ritorna allo step 6 del main scenario
Extension #3 (clicca <i>Canc</i> e poi <i>Annulla</i>)	4d	Clicca <i>Canc</i>	
	5d		Mostra M09
	6d	Clicca <i>Annulla</i>	
	7d		Ritorna allo step 2 del main scenario

3.4.2 Mock-Ups

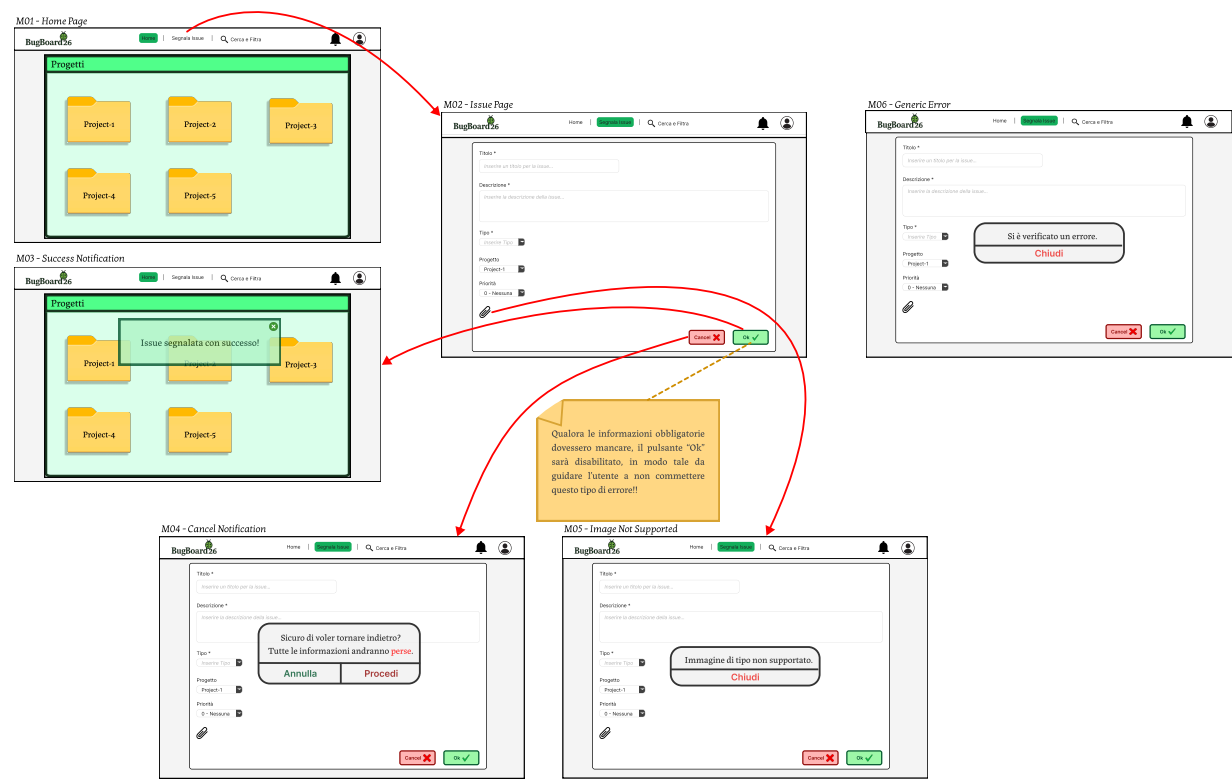


Figura 2: M01-M06

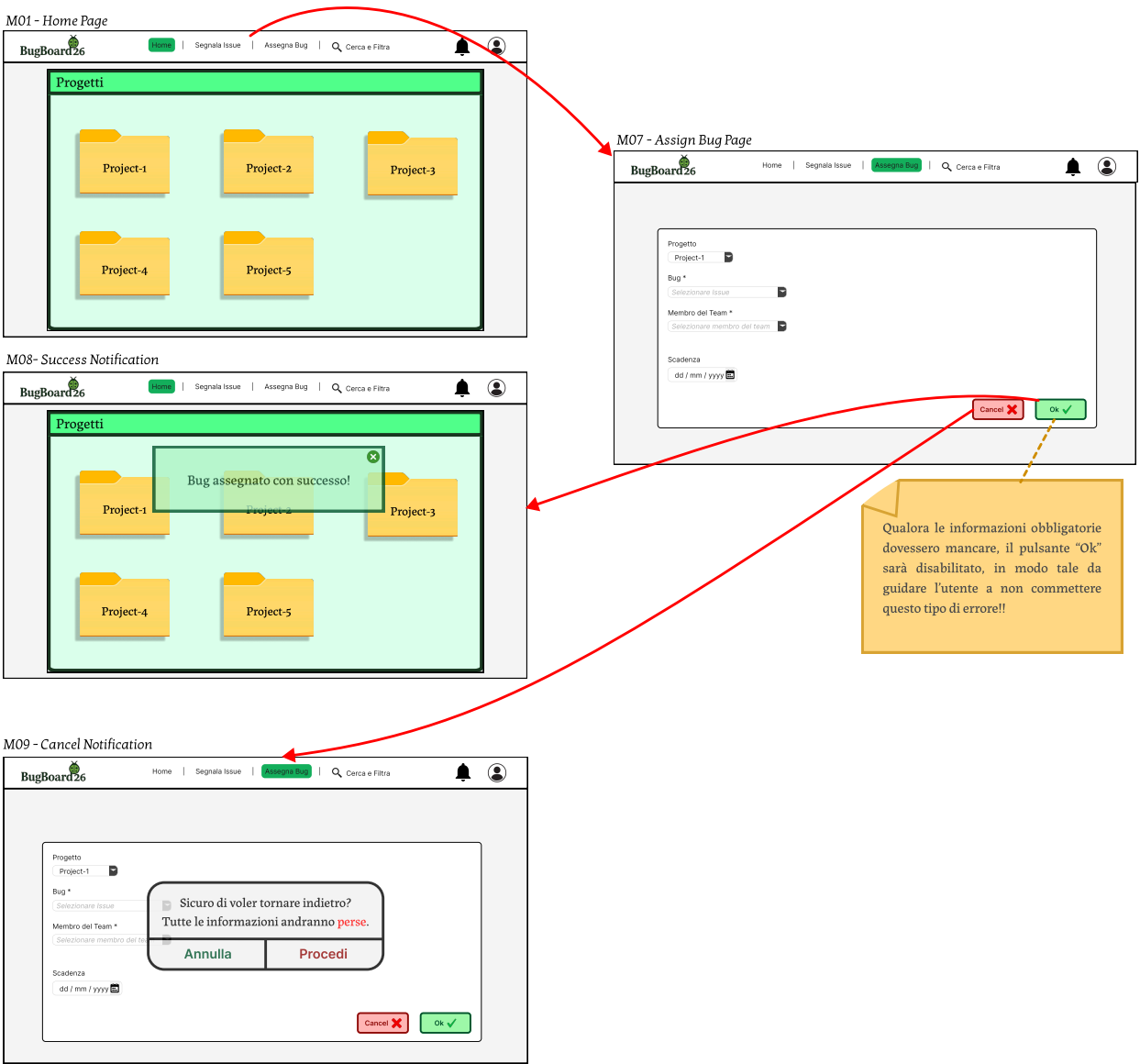


Figura 3: M01, M07-M09

3.5 Modello del Dominio del Problema

Al fine di formalizzare le entità del dominio e le loro relazioni concettuali, viene introdotto il *Modello del Dominio del Problema*.

A differenza dei diagrammi delle classi di progettazione, questo modello astrae totalmente dai dettagli tecnologici e implementativi. Si concentra unicamente sui concetti del mondo reale e sui vincoli di business.

Per gestire la complessità e garantire la massima chiarezza espositiva, la modellazione del dominio verrà presentata in maniera modulare e strutturata per Use Case: per ciascun caso d'uso verrà mostrato e descritto il relativo diagramma concettuale delle entità partecipanti.

3.5.1 Segnalare Issue

In questa prima vista concettuale vengono modellate le entità dal punto di vista dell'attore *Utente*.

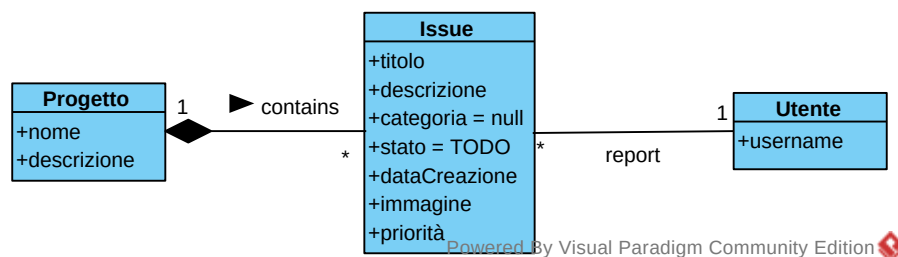


Figura 4: Segnalare Issue

3.5.2 Assegnare Bug

In questa vista vengono modellate le entità necessarie all'Amministratore per delegare la risoluzione di un Bug a un *Utente*, registrando l'eventuale **scadenza** e originando la relativa *Notifica*.

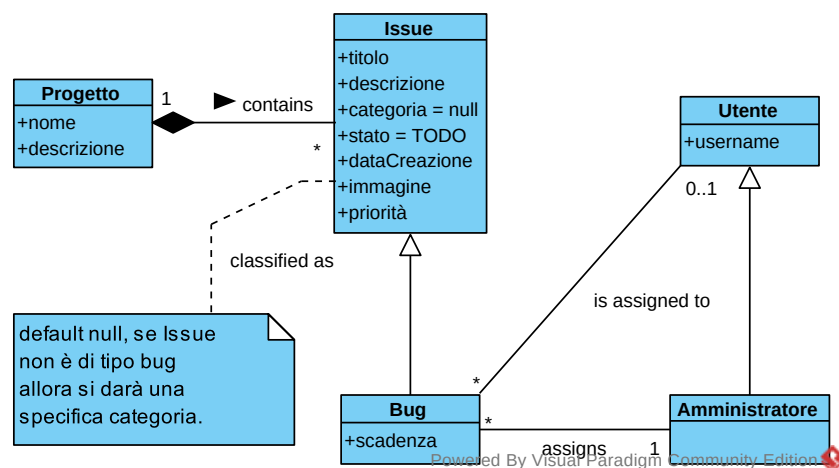


Figura 5: Assegnare Bug

3.5.3 Creare Utenza

Questa vista concettuale modella l'istanziamento di un nuovo account all'interno della piattaforma, definendo le credenziali identificative e il livello di privilegio assegnato alla specifica entità.

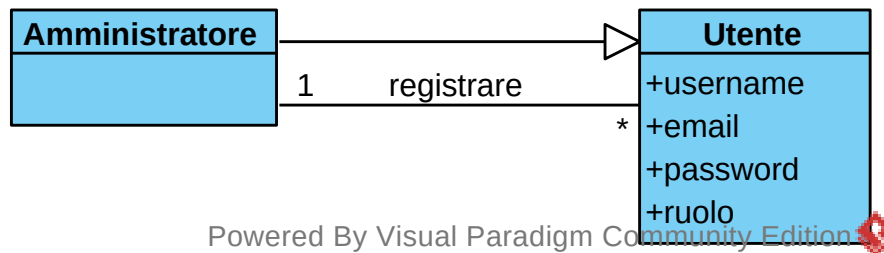


Figura 6: Creare Utenza

3.5.4 Effettuare Login

L'autenticazione all'interno del sistema si traduce, a livello concettuale, nella creazione temporanea di una **SessioneAutenticata**. Tale entità certifica il riconoscimento e abilita l'accesso operativo. Per gestire il ciclo di vita della validità dell'accesso, l'entità prevede l'attributo booleano `scaduta` con valore di default impostato a *false*.

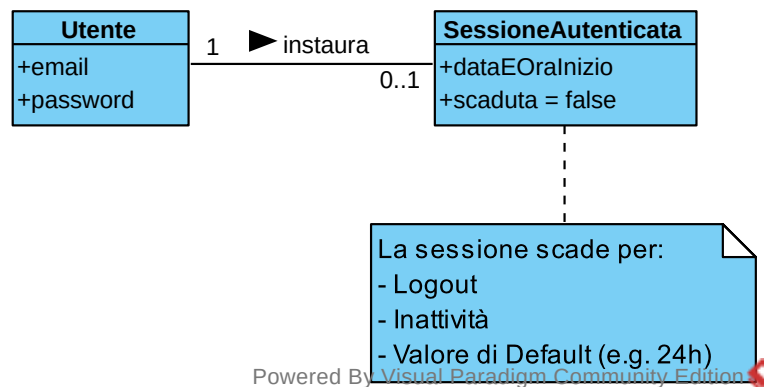


Figura 7: Effettuare Login

3.5.5 Consultare Riepilogo Issue

L'interazione di interrogazione e consultazione dell'elenco globale delle segnalazioni è mediata dalla classe associativa **Ricerca**, che permette di non inquinare le entità principali del dominio. Tale classe incapsula parametri di filtraggio opzionali. L'attributo **testoLibero** garantisce la ricerca per titolo o descrizione, mentre la proprietà **ordinamento** definisce la sequenza di visualizzazione dei risultati, applicando come default l'ordinamento storico crescente basato sulla data di creazione (**CREATED_ASC**).

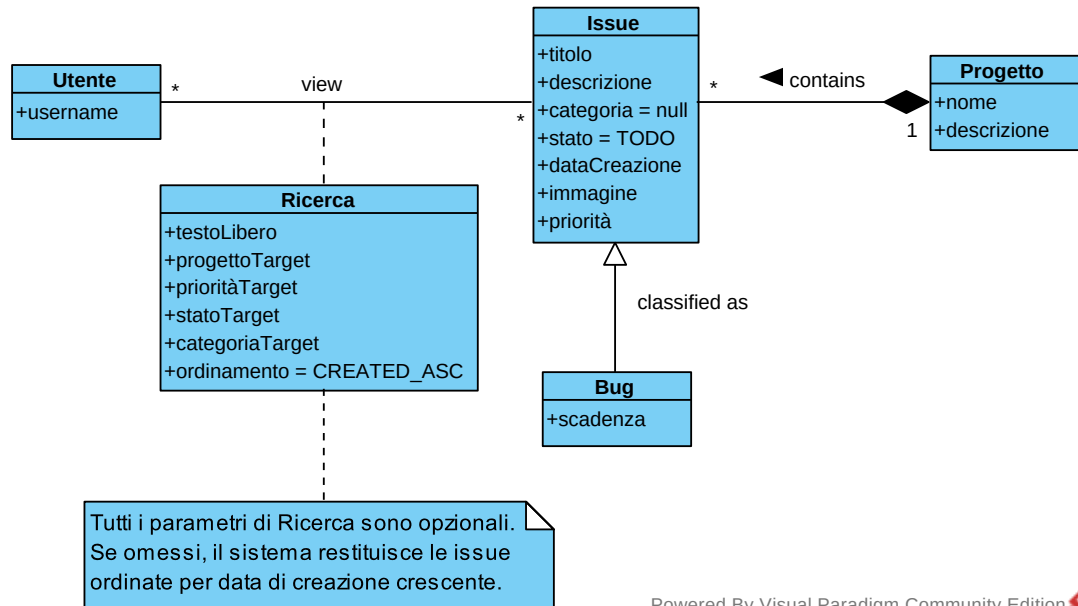


Figura 8: Consultare Riepilogo Issue

3.5.6 Visualizzare Bug Assegnati

La vista evidenzia la prospettiva personale del membro del team, mostrando la relazione con i Bug di cui risulta formalmente responsabile. La classe associativa **Notifica** caratterizza il momento dell'assegnazione, incapsulando il messaggio di avviso, il timestamp di generazione (**dataEOraInvio**) e lo stato di lettura (**letta**), strumenti fondamentali per monitorare i task aperti e le relative scadenze.

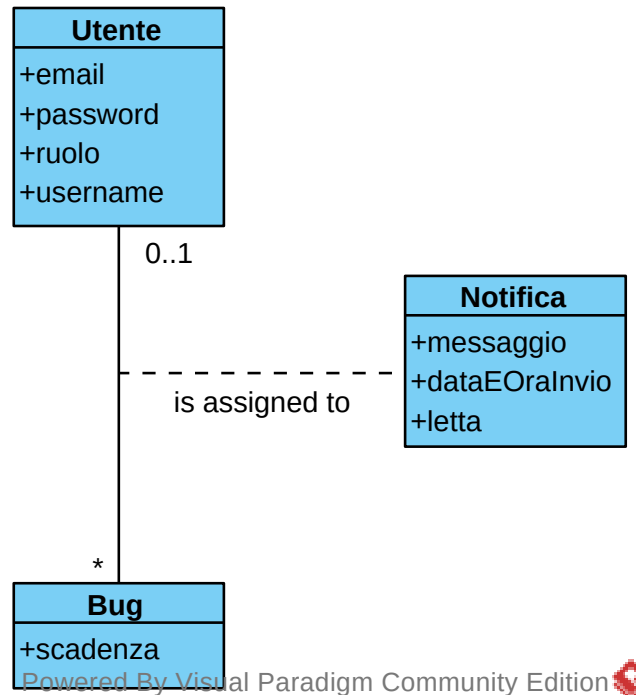


Figura 9: Visualizzare Bug Assegnati

3.5.7 Visualizzare Cronologia Bug

In questa vista viene modellata l'interazione in sola lettura per la consultazione del tracciamento storico delle modifiche subite dai bug.

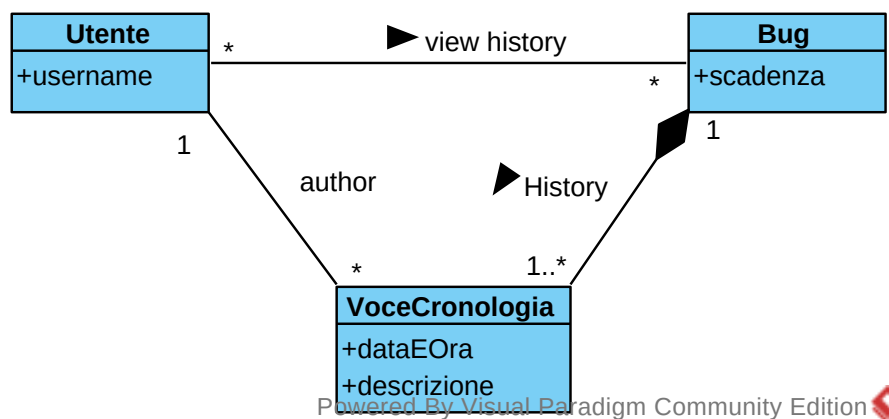


Figura 10: Visualizzare Cronologia Bug

3.5.8 Modificare Bug

Il modello traduce strutturalmente le regole di autorizzazione RBAC in relazioni di molteplicità differenti per la modifica dei dati di un bug. La relazione per l'Utente base impone una molteplicità $0..1 \leftrightarrow *$, assicurando che l'azione di modifica sia consentita esclusivamente sull'eventuale singola identità designata come assegnataria. Di contro, la relazione per l'Amministratore adotta una cardinalità multi-a-molti ($* \leftrightarrow *$), garantendo diritti di alterazione su qualunque istanza presente nel sistema. Per massimizzare l'essenzialità visiva, la superclasse *Issue* è stata intenzionalmente soppressa dal diagramma, mantenendo il focus puro sugli attori e sul Bug.

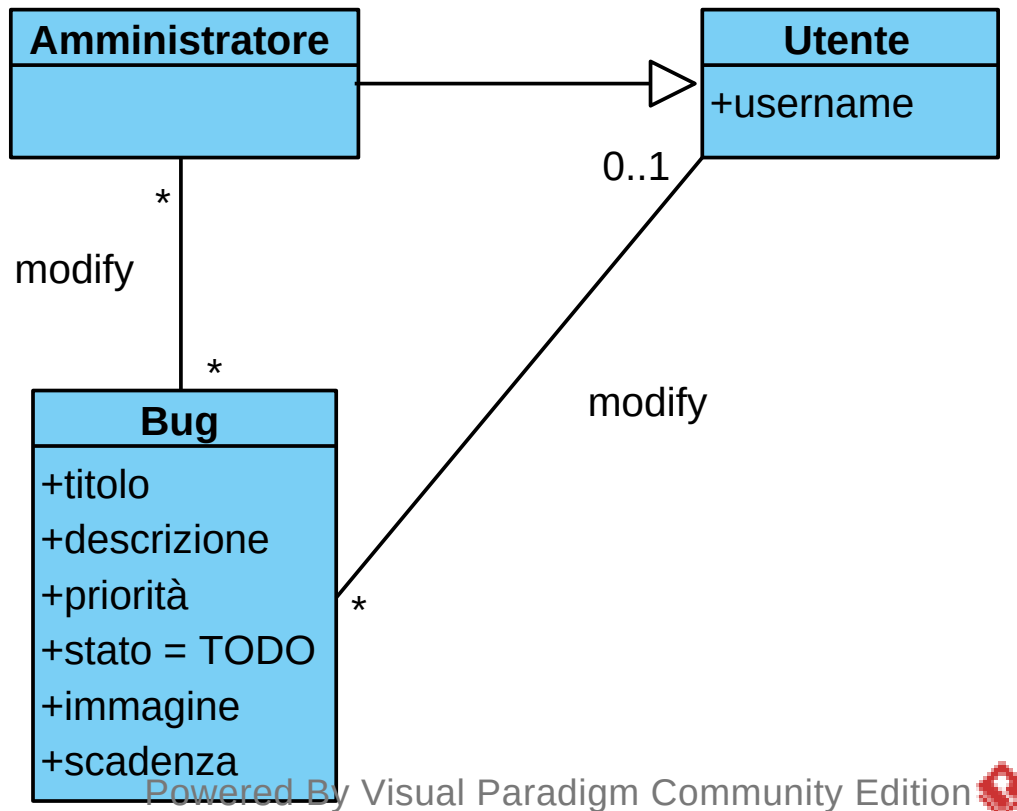


Figura 11: Modificare Bug

3.6 Personas

Al fine di identificare più chiaramente il target utenti di cui si è parlato poc'anzi, si è lavorato con lo strumento dei *Personas*. Nelle pagine seguenti sono riportati due esempi, uno per ogni tipologia di utente.

3.6.1 Persona 1

Roberto Esposito

BIO

Con un approccio lungimirante e determinato, Roberto Esposito lavora come Project Manager presso la KoopaTech, in ambito IT. Laureato all'Università degli Studi di Napoli Federico II, ha intrapreso un percorso professionale che lo ha portato a gestire team e progetti in contesti dinamici e sfidanti. Ora è un leader competente e attento ai dettagli, alla ricerca di nuove tecnologie per rendere più competitiva la sua azienda. Fuori dall'ufficio, è una persona fortemente legata alla famiglia: Roberto, infatti, è felicemente sposato ed è padre di due gemelle, con cui condivide la passione per il cinema.

"Efficienza è la strategia, innovazione è lo strumento."

About

- 48
- Federico II, Napoli
- Napoli, Italia
- Maschio
- Project Manager
- Sposato

Lungimirante Smargiasso Competitivo

FAVORITE BRANDS

- Red Bull
- GitHub
- Udacity
- Napoli

GOALS

- Gestire diversi progetti in contemporanea in modo chiaro.
- Mantenere il controllo sugli account e sui permessi utente.
- Migliorare le performance aziendali.

NEEDS

- Avere una visione complessiva dello stato dei progetti e delle issue aperte.
- Permettere, in maniera sicura, di mostrare i progetti agli stakeholder.
- Monitorare le performance aziendali.

INTERESTS

- Sport: gli piace organizzare partite di calcio con i suoi vecchi compagni.
- Cinema: trova affascinanti i film dell'orrore e i racconti d'autore.
- Famiglia: ritiene importante curare i rapporti all'interno della sua famiglia.

PAIN POINTS

- Mancanza di tracciabilità nelle modifiche effettuate dagli sviluppatori.
- Tempi di attesa proibitivi per operazioni semplici.
- Piattaforme con una curva di apprendimento troppo ripida.

Figura 12: Roberto Esposito, Admin

3.6.2 Persona 2



*"Il codice è classe,
ogni bug è una lezione."*

About

- 22
- Bocconi, Milano
- Milano, Italia
- Maschio
- Junior Developer
- Single

Motivato Impulsivo Empatico

FAVORITE BRANDS



Francesco Sarto

BIO

Francesco Sarto è un giovane sviluppatore appena uscito dall'università con una laurea in Informatica. Appassionato di tecnologia fin da ragazzo, ha iniziato a programmare a 14 anni, creando piccoli progetti personali. Attualmente lavora come junior developer in una startup dove si occupa principalmente di sviluppo back-end con C++ e Java. È sempre alla ricerca di nuove sfide per migliorare le sue competenze e ama partecipare a hackathon e meetup locali. Preferisce lavorare in team e valorizza il feedback dei colleghi più esperti. Nel tempo libero contribuisce a progetti open source, ma è anche un appassionato di videogiochi, tant'è che frequenta fiere a tema, come il Milan Games Week.

GOALS

- Collaborare con il team tramite commenti.
- Monitorare lo stato delle issue assegnate in modo chiaro.
- Contribuire attivamente a progetti.

INTERESTS

- Gaming: appassionato di videogiochi platform e rogue-like.
- UI/UX design: non esperto, ma molto curioso in materia.
- Networking: partecipa regolarmente a meetup tech e conferenze.

NEEDS

- Disponibilità di chat integrata per fornire suggerimenti o richiedere chiarimenti.
- Ricevere notifiche sugli aggiornamenti delle issue assegnate.
- Interfaccia semplice e pronta all'uso.

PAIN POINTS

- Difficoltà nel filtrare e cercare le informazioni che ritiene rilevanti.
- Mancanza di shortcut per ottimizzare processi di routine.
- Strumenti che non garantiscono una comunicazione efficace con i Senior dev.

Figura 13: Francesco Sarto, Developer

3.7 Requisiti Non Funzionali

I Requisiti Non Funzionali definiscono le qualità di servizio, i vincoli architetturali e i criteri di qualità del sistema, in conformità allo standard **ISO/IEC 25010**.

[RNF-01] Usabilità e Accessibilità

L'interfaccia utente web deve essere intuitiva e reattiva. Tutte le operazioni principali devono essere eseguibili entro un massimo di 3 interazioni dall'interfaccia principale. Il sistema deve fornire un feedback visivo immediato per ogni operazione eseguita.

[RNF-02] Prestazioni e Scalabilità

Il sistema deve garantire un tempo di risposta inferiore a 2 secondi per le ricerche, i filtri e per il caricamento delle dashboard contenenti fino a 200 issue. Deve inoltre supportare l'accesso simultaneo di almeno 20 utenti concorrenti senza degrado significativo delle prestazioni.

[RNF-03] Sicurezza Applicativa

L'accesso al sistema richiede autenticazione basata su email e password. Le password devono essere memorizzate nel database in forma cifrata tramite algoritmi di hashing sicuri. Le comunicazioni client-server avvengono mediante API REST cifrate e il sistema adotta le linee guida OWASP contro attacchi XSS, CSRF e SQL Injection.

[RNF-04] Architettura Distribuita

Il codice sorgente deve rispettare la netta separazione delle responsabilità tra logica di visualizzazione e di business, organizzando il sistema per facilitare l'esecuzione di test unitari automatizzati e la scalabilità dei componenti.

[RNF-05] Compatibilità

L'applicazione web deve funzionare correttamente sui principali browser moderni come Chrome, Firefox e Safari. Il caricamento di allegati per le issue deve supportare i formati immagine più comuni, con un limite dimensionale di 5MB per singolo file.

3.8 Requisiti e Vincoli di Dominio

I Requisiti e Vincoli di Dominio comprendono le regole operative essenziali per il funzionamento della piattaforma e i vincoli normativi legati al mercato di distribuzione del software.

Regole Operative del Dominio

[RD-01] Cronologia delle Modifiche

Ogni bug deve possedere una cronologia che registri in modo immutabile l'identità dell'autore del cambiamento e il timestamp esatto, partendo obbligatoriamente dall'evento di creazione iniziale.

[RD-02] Classificazione e Stato Iniziale

Ogni issue deve essere classificata secondo una tipologia predefinita. Al momento della creazione, lo stato della segnalazione assume inderogabilmente il valore di partenza **todo**.

Vincoli Normativi di Dominio e Conformità Territoriale

[RD-03] Conformità alla Protezione Dati

Per consentire la distribuzione e la commercializzazione del software all'interno dell'Unione Europea, il sistema deve sottostare tassativamente al regolamento GDPR. La raccolta

dei dati personali avviene solo previo consenso esplicito, garantendo l'accesso, la rettifica e la cancellazione tramite il diritto all'oblio. Il mancato rispetto di tali direttive legali territoriali costituisce vincolo bloccante per la messa in produzione.

[RD-04] Unicità e Validazione delle Identità Utente

Ogni indirizzo e-mail registrato nel sistema deve essere univoco nell'ambito del database applicativo per prevenire duplicazioni di utenze e furti di identità. Eventuali collisioni devono impedire il completamento della registrazione fornendo immediata diagnostica.

3.9 Assunzioni e dipendenze

Lo sviluppo e il corretto funzionamento del sistema dipendono dalle seguenti condizioni preliminari:

- disponibilità di una connessione di rete stabile per il trasporto dei dati tra client e strato server;
- presenza di un'infrastruttura in grado di ospitare e orchestrare l'applicativo back-end e la sua persistenza;
- utilizzo vincolante di tool di progettazione CASE per la documentazione visiva;
- utilizzo di un sistema di versioning remoto per la storicizzazione del codice e degli artefatti;
- eventuali integrazioni con servizi terzi per l'erogazione delle comunicazioni email avverranno tramite API autenticate.